

Supplement: N -level hierarchical POMDP methods

This supplement specifies the generic N -level architecture that sec. 14.11 exercises at depth three: how the meta-context (L3), context (L2), and location (L1) factors are chained through conditioned priors,

Supplement: N-level... (part 2)

what the declarative `LayerSpec` interface fixes versus leaves free, and the top-down/bottom-up passes the inference runs.

Supplement: N-level... (part 3)

It answers *what the executed 3-level result is a special case of* — the reason the same code runs at other depths without new mathematics — while recording that only the declared 3-level configuration is empirically evaluated here.

Generative model for an N-level hierarchy

The 3-level POMDP is built by `build_3level_world` in the `fedference.pomdp` module. It extends the 2-level construction (sec. 14.10) by adding a top-level meta-context factor.

Generative model for an... (part 2)

L3 (meta-context). 2 low_threat / high_threat states begin from a uniform prior and gate the L2 context prior.

L2 (context). 2 quiet / alert states gate the L1 prior through context-conditioned location priors.

Generative model for an... (part 3)

L1 (location). The standard 3x3 grid has 9 states and sensor acuity 0.85.

The conditioned priors are (see eq. 38 and eq. 39):

Generative model for an... (part 4)

L3 state	L2 prior (quiet, alert)
low_threat	(0.50, 0.50) — uniform context
high_threat	(0.20, 0.80) — peaked at alert

Generative model for an... (part 5)

L2 state	L1 prior
quiet	uniform over all 9 location states
alert	mass 0.60 at center cell (flat index 4), residual uniform

Generic N-level architecture

The `LayerSpec` type and `build_nlevel_world` function in the `fedference.pomdp` module implement the generic N-level version. The declarative layer specification is `hierarchical_layers.yaml` under `src/fedference/config/`;

Generic N-level architecture (part 2)

it mirrors the canonical 3-level defaults (a standalone documentation artifact not read by any code path, kept in sync with the `build_3level_world` defaults).

Generic N-level architecture (part 3)

The constructor accepts $\text{depth} \geq 2$; the executed empirical result in this manuscript is restricted to the declared 3-level configuration, and the leaf layer must carry `n_states == N_LOCATIONS`.

Inference algorithm across hierarchy levels

`fedference.pomdp.nlevel_infer` performs 4 passes of top-down / bottom-up alternating minimization over all N levels:

Inference algorithm across... (part 2)

1. Top-down pass. Compute the empirical prior for each level by marginalizing over the level above (eq. 38, eq. 39).

Inference algorithm across... (part 3)

2. L1 update. The one-step variational posterior on the observation is

$$q_{\text{loc}} = \text{softmax}(\log \tilde{\pi}_{0, \text{L1}} + \log A[\text{obs}, \cdot]).$$

Inference algorithm across... (part 4)

3. Bottom-up pass. Update each non-leaf level's belief from the marginal evidence contributed by the level below:

$$\ell_j = \log(\tilde{p}_{\text{child}|\text{parent}=j}^\top q_{\text{child}}).$$

Inference algorithm across... (part 5)

After 4 iterations the agent broadcasts all N level beliefs; the colony federates each level independently via a log-linear pool (eq. 9).

Study parameters for the three-level run

Parameter	Value
Agents	4
Trials	960
Acuity	0.85

Study parameters for the... (part 2)

The executed three-level configuration is summarized in tbl. 18.